Object-Oriented Programming

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)

COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY

Asabhya Maanav

2026 EDITION

VERSION 1.0

Table of Contents

Object-Oriented Programming (Java) — Complete Notes

This guide builds Object-Oriented Programming concepts progressively for the **IBPS SO (IT Officer)** examination, beginning with paradigms and ending with multithreading, event handling and applets. Every fact is exam-oriented and MCQ-ready. The reference language throughout is **Java**.

1. OOP Concepts & Fundamentals

1.1 Programming Paradigms

A **programming paradigm** is a style or approach of writing programs. The two paradigms compared in the syllabus are **Procedural** (process-oriented) and **Object-Oriented** programming.

1.1.1 Procedural vs Object-Oriented Programming

Procedural programming (e.g. **C**, Pascal, FORTRAN) decomposes a program into **functions/procedures** that operate on data. Data and functions are **separate**, and data largely moves **freely** through global functions. **Object-Oriented programming (OOP)** (e.g. **Java**, C++, Python) decomposes a program into **objects** that bundle **data + behaviour** together, exposing only what is necessary.

Basis	Procedural	Object-Oriented
Basic unit	**Function/procedure**	**Object** and **class**
Approach	**Top-down**	**Bottom-up**
Data + code	**Separate**	**Bound together** (encapsulation)
Data access	Mostly **global**, less secure	**Hidden** via access modifiers, secure
Reusability	Limited	High via **inheritance**
Examples	**C**, Pascal	**Java**, C++

1.1.2 Need and Advantages of OOP

- **Modularity:** code is organised into self-contained objects.
- **Reusability:** existing classes are reused via **inheritance** and composition.
- **Data security / data hiding:** achieved by **encapsulation** and **access modifiers**.
- **Maintainability & extensibility:** changes are localised to objects.
- **Polymorphism:** one interface, many implementations, reduces complexity.
- **Real-world modelling:** objects map naturally to real entities.

Memory Trick: OOP gives **A-PIE** of benefits backed by the **4 pillars**: **A**bstraction, **P**olymorphism, **I**nheritance, **E**ncapsulation.

1.2 Core OOP Principles (Overview)

The **4 pillars** of OOP are **Abstraction, Encapsulation, Inheritance, Polymorphism**. Some texts add **Class/Object** and **Message Passing** as supporting concepts.

1.2.1 Class and Object

A **class** is a **blueprint / template / logical entity** that defines state (fields) and behaviour (methods). An **object** is a **real-world instance** of a class — it occupies **memory at runtime**. A class is created **once**; many objects can be instantiated from it.

1.2.2 Abstraction

Abstraction means showing **only essential features** and **hiding background details** (the "what", not the "how"). Achieved in Java by **abstract classes** and **interfaces**. Example: pressing a car's accelerator hides the internal combustion logic.

1.2.3 Encapsulation

Encapsulation is **wrapping data and code together** into a single unit (class) and **restricting direct access** using access modifiers — the **data-hiding** mechanism. Achieved by making fields **private** and exposing **public getters/setters**.

1.2.4 Inheritance

Inheritance lets a **child (subclass)** acquire properties and behaviour of a **parent (superclass)**, promoting **code reuse**. Keyword: **extends**. Establishes an **IS-A** relationship.

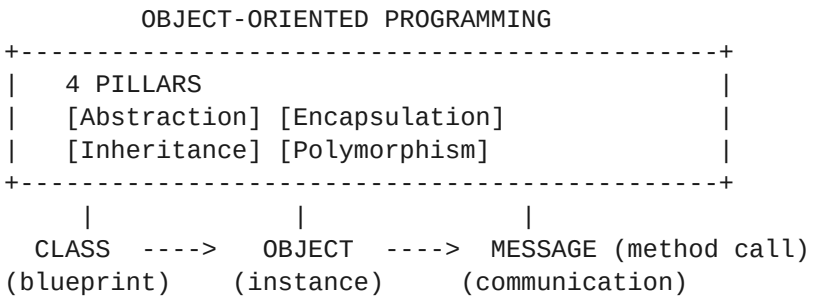
1.2.5 Polymorphism

Polymorphism ("many forms") lets one entity behave differently in different contexts: **compile-time** (method **overloading**) and **runtime** (method **overriding**).

1.2.6 Message Passing

Message passing means objects communicate by **invoking methods** on one another. A message includes the **target object**, the **method name**, and the **arguments**.

> **MCQ Trap:** Abstraction hides **implementation/complexity**; Encapsulation hides **data**. Abstraction is achieved with abstract classes/interfaces; encapsulation with **private fields + getters/setters**. Examiners swap these definitions.



2. Java Basics

2.1 Introduction

2.1.1 History of Java

- Developed by **James Gosling** and his team (the "**Green Team**") at **Sun Microsystems**.
- Project started in **1991**; the language was first named **Oak** (after an oak tree outside Gosling's office), later **Greentalk**, and finally renamed **Java** in **1995**.
- First public release **JDK 1.0** in **1995** (officially **1996**). Original slogan: **"Write Once, Run Anywhere" (WORA)**.
- **Sun Microsystems** was acquired by **Oracle Corporation** in **2010**, which now owns Java.

2.1.2 Features of Java (Buzzwords)

The classic Sun buzzwords are: **Simple, Object-Oriented, Platform Independent, Secured, Robust, Architecture-Neutral, Portable, Dynamic, Interpreted, High Performance, Multithreaded, Distributed**.

- **Platform Independent:** source compiles to **bytecode** (`.class`), which runs on any **JVM** — "**Write Once, Run Anywhere**".
- **Robust:** strong memory management, **automatic garbage collection**, exception handling, no explicit pointers.
- **Secure:** no pointers, runs inside the **JVM sandbox**, bytecode verifier.
- **Multithreaded:** built-in support for concurrent threads.
- **Distributed:** supports RMI and networking (`java.net`).

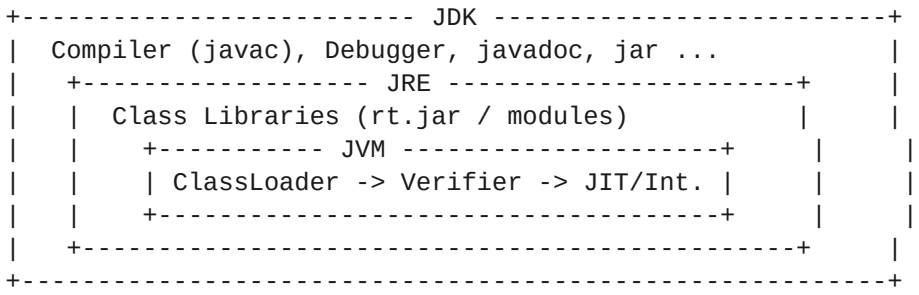
MCQ Trap: Java is **platform independent** but the **JVM is platform dependent** — a different JVM exists for each OS. Java is **NOT** purely compiled or purely interpreted; it is **both compiled (to bytecode) and interpreted (by JVM)**.

2.1.3 JDK, JRE, JVM Architecture

- **JVM (Java Virtual Machine):** abstract machine that **executes bytecode**; it is **platform dependent**. Provides the runtime environment.
- **JRE (Java Runtime Environment): JVM + core libraries**; needed to **run** Java programs.
- **JDK (Java Development Kit): JRE + development tools** (`javac`, `java`, `jar`, `javadoc`, `jdb`); needed to **develop** Java programs.

Component	Contains	Purpose
JVM	Class loader, bytecode verifier, JIT, interpreter	**Executes** bytecode
JRE	**JVM + libraries**	**Runs** applications
JDK	**JRE + compiler/tools**	**Develops** applications

Memory Trick: JDK > JRE > JVM (each fully contains the next). "**D**evelop > **R**un > e**X**ecute".



2.1.4 Structure of a Simple Java Program & Execution

A minimal program defines a **class** with a `main` method:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, IBPS SO!");
    }
}
```

- The `main` signature is `public static void main(String[] args)` — `public` (JVM access), `static` (no object needed), `void` (no return), `String[] args` (command-line arguments).
- The **file name must match the public class name** (`Hello.java`).

Compilation & execution process:

- Compile: `javac Hello.java` -> produces `Hello.class` (**bytecode**).
- Run: `java Hello` -> the **JVM** loads, verifies, and executes the bytecode.

```
Hello.java --(javac)--> Hello.class --(java/JVM)--> Output
 source         compiler   bytecode      runtime
```

MCQ Trap: `javac` invokes the **compiler**; `java` invokes the **JVM/launcher**. The class file holds **bytecode**, not machine code. A `.java` file may have **multiple classes** but **at most one public class**, whose name must match the file.

2.2 Data Types & Variables

2.2.1 Primitive Data Types

Java has **8 primitive data types**. Sizes are **fixed and platform independent**.

Type	Size	Default	Range
byte	**8 bits (1 B)**	**0**	**-128 to 127**
short	**16 bits (2 B)**	**0**	-32768 to 32767
int	**32 bits (4 B)**	**0**	-2^31 to 2^31-1
long	**64 bits (8 B)**	**0L**	-2^63 to 2^63-1
float	**32 bits (4 B)**	**0.0f**	~6-7 decimal digits
double	**64 bits (8 B)**	**0.0d**	~15 decimal digits
char	**16 bits (2 B)**	**\u0000**	**0 to 65535** (Unicode)
boolean	**JVM-dependent (1 bit logical)**	**false**	**true / false**

MCQ Trap: In Java **char is 2 bytes** (Unicode, **unsigned**), unlike C where it is 1 byte. **boolean** has **no defined size** (cannot be cast to int). The **smallest** integer type is **byte**; the **largest** is **long**.

2.2.2 Non-Primitive (Reference) Types

Non-primitive types include **classes, interfaces, arrays, String**. They store a **reference (address)**, default to **null**, and are created with **new** (except String literals and arrays via `{}`).

2.2.3 Variables — Declaration, Initialization, Types

A **variable** is a named memory location. Java has **3 kinds**:

- **Local variables:** declared inside methods/blocks; **no default value**; must be initialized before use; stored on the **stack**.
- **Instance variables:** declared in a class but outside methods; **one copy per object**; get **default values**; stored on the **heap**.
- **Static (class) variables:** declared with **static**; **one shared copy** for all objects; stored in the **method area**.

2.2.4 Scope and Lifetime of Variables

- **Scope** = region where a variable is **accessible**. **Lifetime** = how long it stays in **memory**.
- **Local:** scope and lifetime limited to the **block/method**.
- **Instance:** lives as long as the **object** lives.
- **Static:** lives for the **entire program** (until class unloads).

2.2.5 Constants and Literals

- A **constant** is a fixed value declared with **final** (e.g. `final int MAX = 100;`).
- **Literals** are fixed source-code values: **integer** (`10`, `0x1F` hex, `012` octal, `0b101` binary), **floating** (`3.14`, `2e3`), **char** (`'A'`), **String** (`"hi"`), **boolean** (`true`/`false`), and `null`.

2.2.6 Type Conversion and Casting

- **Widening (implicit / automatic):** smaller -> larger, **no data loss**, done by the compiler. Order: **byte -> short -> int -> long -> float -> double**; and **char -> int**.
- **Narrowing (explicit casting):** larger -> smaller; **may lose data**; requires a **cast** `(type)`.

```
WIDENING (automatic, safe)  ----------------------->
byte -> short -> int -> long -> float -> double
char ----------^
<-----------------------  NARROWING (explicit cast)
```

MCQ Trap: Widening is automatic; **narrowing needs an explicit cast**. `char` widens to `int` but `byte`/`short` do **not** widen to `char` automatically. Mixing an `int` with a `double` promotes the result to **double**.

2.3 Operators & Expressions

Java operators by category:

- **Arithmetic:** `+ - * / %` (modulus). `+` is also **String concatenation**.
- **Relational:** `== != > < >= <=` (result is **boolean**).
- **Logical:** `&&` (short-circuit AND), `||` (short-circuit OR), `!` (NOT).
- **Bitwise:** `& | ^ ~`, shifts `<<`, `>>` (signed), `>>>` (unsigned right shift).
- **Assignment:** `=` and compound `+= -= *= /= %=`.
- **Unary / Increment-Decrement:** `++ --` (prefix vs postfix), unary `+ - !`.
- **Ternary:** `condition ? expr1 : expr2` (the **only 3-operand** operator).

2.3.1 Operator Precedence & Associativity

- **Precedence** decides **which operator binds first**; **associativity** decides direction when precedence ties.
- Highest to lowest (simplified): **postfix `++/--` > unary > `* / %` > `+ -` > shifts > relational > equality > bitwise `& ^ |` > logical `&& ||` > ternary `?:` > assignment**.
- Most operators are **left-to-right** associative; **assignment** and **ternary** are **right-to-left**.

> **MCQ Trap:** `&&`/`||` are **short-circuit** (stop evaluating once result is known); `&`/`|` evaluate **both operands**. `>>>` (unsigned shift) **fills with 0**; `>>` preserves the **sign bit**.

2.4 Control Statements

- **Selection:** `if`, `if-else`, **nested if**, `if-else-if` ladder, **switch**.
- **Iteration (loops):** `for`, `while`, `do-while` (**executes at least once**), enhanced **for-each** (`for(T x : arr)`).
- **Jump:** `break` (exit loop/switch), `continue` (skip to next iteration), `return` (exit method); labelled `break`/`continue`.

The **switch** statement accepts **byte, short, int, char, String (Java 7+), enum**, and wrapper types — but **NOT long, float, double, or boolean**.

> **MCQ Trap:** `do-while` runs **at least once** (condition checked **after** body). `switch` **does not allow** `long`/`float`/`double`/`boolean`. Missing `break` in a `switch` causes **fall-through**.

2.5 Arrays

An **array** is a **fixed-size**, **indexed** (**0-based**) collection of elements of the **same type**, stored **contiguously**. Arrays are **objects** in Java with a `length` **field** (not a method).

- **One-dimensional:** `int[] a = new int[5];` or `int[] a = {1,2,3};`.
- **Two-dimensional:** `int[][] m = new int[3][4];` (array of arrays).
- **Jagged array:** rows of **unequal length**, e.g. `int[][] j = new int[3][];`.
- **Array of objects:** `Student[] s = new Student[10];` (each element defaults to **null**).

```
int[] a = {10, 20, 30, 40};
 index:   0    1    2    3      <- 0-based
 a.length = 4   (a FIELD, not a method)
```

> **MCQ Trap:** Use `arr.length` (**field**, arrays), `str.length()` (**method**, String), and `list.size()` (**method**, Collection). Array index out of bounds throws `ArrayIndexOutOfBoundsException` (unchecked). Array size is **fixed** after creation.

3. Classes, Objects & Methods

3.1 Classes and Objects

A **class** groups **fields (state)** and **methods (behaviour)**. An **object** is created with the `new` operator, which allocates **heap** memory and returns a **reference**.

```
class Student {            // class = blueprint
    int roll;              // instance variable
    void show() { ... }    // instance method
}
Student s = new Student();  // object on the heap; s = reference
```

- **Instance variables** belong to each object; **instance methods** operate on object state via the implicit `this` reference.
- Multiple objects each get their **own copy** of instance variables.

3.2 Constructors

A **constructor** initializes a new object. Rules: **same name as the class**, **no return type** (not even `void`), called **automatically** by `new`.

- **Default constructor:** if **no constructor** is defined, the **compiler inserts** a no-argument constructor that sets **default values**.
- **No-argument constructor:** explicitly written, takes no parameters.
- **Parameterized constructor:** accepts arguments to initialize fields.
- **Copy constructor:** Java has **NO built-in copy constructor** (unlike C++); you must write one manually that takes another object.
- **Constructor overloading:** multiple constructors with **different parameter lists**.

> **MCQ Trap:** Constructors have **no return type** and are **not inherited**. If you define **any** constructor, the compiler **no longer** supplies the default one. Java provides **no built-in copy constructor** and **no destructor** (cleanup is via **garbage collection**).

3.3 Methods

A **method** is a named block of code. Declaration: `[modifiers] returnType name(parameters) { body }`. Called on an object (instance) or class (static).

3.3.1 Method Overloading

Method overloading = same method **name** with a **different parameter list** (number, type, or order). It is **compile-time (static) polymorphism**. **Return type alone does NOT** distinguish overloaded methods.

3.3.2 Constructor Overloading

Defining **multiple constructors** with different parameter lists; `this(...)` invokes one constructor from another (**must be the first statement**).

3.3.3 Parameter Passing (Call by Value / Reference)

Java is **strictly pass-by-value**. For **primitives**, the **value** is copied. For **objects**, the **value of the reference** is copied — so the called method can **modify the object's state** but **cannot reassign** the caller's reference.

> **MCQ Trap:** Java is **always pass-by-value** — there is **no pass-by-reference**. For objects it copies the **reference value**, which only *looks* like pass-by-reference.

3.3.4 Recursion

Recursion = a method **calling itself**. Needs a **base case** to terminate, else `StackOverflowError`. Each call adds a **stack frame**.

```
long fact(int n) {
    if (n <= 1) return 1;     // base case
    return n * fact(n - 1);   // recursive call
}
```

3.4 Keywords & Members

3.4.1 The `this` Keyword

`this` is a reference to the **current object**. Uses: distinguish **instance variable from parameter** of the same name, call another **constructor** `this()`, pass the current object, return the current object.

3.4.2 The `static` Keyword

- **static variable: one shared copy** for the whole class (class variable).
- **static method:** belongs to the class; **cannot use** `this`/`super` or access **instance** members directly; `main` is static.
- **static block:** runs **once** when the class is **loaded** (before `main`).
- **static nested class:** does not need an outer instance.

3.4.3 The `final` Keyword

- **final variable:** a **constant** (value cannot change after assignment).
- **final method: cannot be overridden**.
- **final class: cannot be inherited/extended** (e.g. **String**, all wrapper classes).

> **MCQ Trap:** Distinguish `final` (constant / no override / no inherit), `finally` (block that **always executes** in exception handling), and `finalize()` (method called by **GC** before destroying an object). These three are repeatedly confused.

3.4.4 Access Control (Access Modifiers)

Java has **4 access levels** ordered most-to-least restrictive: **private < default (package-private) < protected < public**.

Modifier	Same Class	Same Package	Subclass (other pkg)	Other Package
private	**Yes**	No	No	No
default	Yes	**Yes**	No	No
protected	Yes	Yes	**Yes**	No
public	Yes	Yes	Yes	**Yes**

> **Memory Trick:** Visibility widens **Private -> Default -> Protected -> Public** ("**Pa Di Pro Pu**"). A **top-level class** can be only **public** or **default** (not private/protected).

3.4.5 Garbage Collection & `finalize()`

- **Garbage collection (GC) automatically reclaims** memory of objects with **no live references**. Java has **no manual** `free()`/`delete`.
- `System.gc()` only **requests** GC (no guarantee). `finalize()` (deprecated since **Java 9**) was called **once** by the GC before reclaiming an object.
- Eligibility: nulling a reference, reassigning, or object going **out of scope**.

3.5 Nested and Inner Classes

A **nested class** is a class declared inside another class. **4 types**:

- **Static nested class:** declared **static**; accesses only the outer class's **static** members; no outer instance needed.
- **(Non-static) Inner class:** tied to an **outer instance**; can access **all** outer members including private.

- **Local inner class:** defined **inside a method**; scope limited to that method.
- **Anonymous inner class:** a class with **no name**, declared and instantiated at once — commonly used for **listeners** and **Runnable**.

3.6 String Handling

3.6.1 String Class & Immutability

- **String** objects are **immutable** — once created, their value **cannot change**; any modification creates a **new object**.
- **String literals** are stored in the **String Constant Pool (SCP)** inside the heap; identical literals **share** the same object. `new String("x")` always creates a **new heap object**.
- Common methods: `length()`, `charAt()`, `substring()`, `concat()`, `indexOf()`, `equals()`, `equalsIgnoreCase()`, `compareTo()`, `toUpperCase()`, `trim()`, `replace()`, `split()`.

3.6.2 StringBuffer and StringBuilder

Both are **mutable** sequences of characters.

Feature	String	StringBuffer	StringBuilder
Mutability	**Immutable**	**Mutable**	**Mutable**
Thread-safe	Yes (immutable)	**Yes (synchronized)**	**No**
Performance	Slow on edits	Slower (sync)	**Fastest**
Since	1.0	1.0	**Java 5**

MCQ Trap: String is immutable; **StringBuffer is synchronized (thread-safe)** but slower; **StringBuilder is not synchronized** but faster. Use `==` for **reference** comparison and `equals()` for **content** comparison of strings.

4. Inheritance, Polymorphism, Encapsulation

4.1 Inheritance

Inheritance lets a subclass reuse and extend a superclass using `extends`. The root of **all** classes is `java.lang.Object`. It models an **IS-A** relationship.

4.1.1 Types of Inheritance

- **Single:** one subclass, one superclass.
- **Multilevel:** a chain A -> B -> C.
- **Hierarchical:** many subclasses share one superclass.
- **Multiple: NOT supported with classes** (avoids the **diamond problem**); achieved only via **interfaces**.
- **Hybrid:** combination; with classes it is not allowed, only via interfaces.

```
Single        Multilevel       Hierarchical
  A             A                  A
  |             |                / | \
  B             B               B  C  D
                |
                C
```

4.1.2 `super` Keyword & Constructor Chaining

- `super` refers to the **immediate parent**: `super.method()`, `super.field`, and `super(...)` calls the parent constructor.
- **Constructor chaining:** subclass constructors **implicitly call** `super()` **first**; constructors execute **top-down** (parent before child).

4.1.3 Method Overriding

Overriding = subclass provides a **new definition** of an inherited method with the **same signature**. Rules: **same name + parameters**; return type same or **covariant**; access **cannot be more restrictive**; `final`/`static`/`private` methods can't be overridden; use `@Override`.

4.2 Polymorphism

Polymorphism = one name, many forms.

- **Compile-time (static) polymorphism: method overloading**; resolved by the **compiler** using **early/static binding**.
- **Runtime (dynamic) polymorphism: method overriding**; resolved by the **JVM** at runtime using **dynamic method dispatch** (**late binding**) based on the **actual object** type.

```
Animal a = new Dog();   // reference Animal, object Dog
a.sound();              // runs Dog's sound()  -> DYNAMIC DISPATCH
```

MCQ Trap: Overloading = compile-time = static binding; **Overriding = runtime = dynamic binding**. A superclass **reference** can hold a subclass **object** (upcasting), and the **overridden** method that runs depends on the **object**, not the reference.

4.2.1 Overloading vs Overriding

Basis	Overloading	Overriding
Binding	**Compile-time (static)**	**Runtime (dynamic)**
Classes	**Same class**	**Two classes (IS-A)**
Parameters	**Must differ**	**Must be same**
Return type	Can differ	**Same or covariant**
Polymorphism	**Static**	**Dynamic**

4.3 Encapsulation

Encapsulation binds data and methods into a class and **hides data** by declaring fields **private**, exposing controlled access through **public getters and setters**. Benefits: **data hiding, control, flexibility, reusability, testability**.

```
class Account {
    private double balance;                 // hidden data
    public double getBalance(){return balance;}        // getter
    public void setBalance(double b){ if(b>=0) balance=b; } // setter with validation
}
```

4.4 Abstraction

Abstraction exposes **essential behaviour** while hiding implementation, achieved by:

- **Abstract class:** declared with `abstract`; **cannot be instantiated**; may contain **both abstract (no body) and concrete methods**, constructors, and instance fields. A class with **at least one abstract method must be abstract**.
- **Interface:** a fully abstract contract (see Unit 5).

MCQ Trap: You **cannot create an object** of an abstract class, but it **can have a constructor** (called via subclass). An abstract method has **no body** and **must be overridden** by the first concrete subclass.

5. Packages & Interfaces

5.1 Packages

A **package** is a **namespace** that groups related classes/interfaces, preventing **name conflicts** and controlling **access**.

- Declared with `package pkg.name;` as the **first statement** in a file.
- Imported with `import pkg.ClassName;` or `import pkg.*;`.
- `java.lang` is **imported automatically** (contains String, Object, System, Math, wrappers, Thread).

5.1.1 Built-in Packages

- `java.lang` — core classes (auto-imported).
- `java.util` — Collections, Scanner, Date, Random.
- `java.io` — input/output streams, File.
- `java.net` — networking; `java.awt` — GUI; `javax.swing` — Swing GUI; `java.sql` — JDBC.

5.1.2 CLASSPATH & Access Protection

- **CLASSPATH** is the environment setting telling the JVM/compiler **where to find** `.class` files and JARs.
- Across packages, only **public** and **protected** (in subclasses) members are accessible; **default** members are **package-private**.

5.2 Interfaces

An **interface** is a **fully abstract contract**. By default its **methods are** `public abstract` and its **variables are** `public static final` **(constants)**.

- A class **implements** an interface with `implements` and must override all abstract methods.
- A class can implement **multiple interfaces** -> this gives Java **multiple inheritance of type**.
- **Java 8+** added `default` methods (with body) and `static` methods; **Java 9+** added **private** methods.
- **Marker interface:** has **no methods** (e.g. **Serializable**, **Cloneable**).
- **Functional interface (Java 8):** exactly **one abstract method** (e.g. **Runnable**, **Comparable**), usable with **lambdas**; marked `@FunctionalInterface`.

5.2.1 Interface vs Abstract Class

Basis	Abstract Class	Interface
Keyword	**abstract class**	**interface**
Methods	Abstract **+** concrete	Abstract (+ default/static in **Java 8+**)
Variables	Any kind	**public static final** only
Constructor	**Yes**	**No**
Inheritance	**Single (extends)**	**Multiple (implements)**
Access modifiers	Any	Methods implicitly **public**
Use	Partial abstraction + shared state	Full abstraction / contract

MCQ Trap: An interface gives **100% abstraction** (pre-Java 8); an abstract class gives **partial** abstraction. Interface variables are **always** `public static final`. A class **extends one** class but **implements many** interfaces.

6. Exception Handling

An **exception** is an **abnormal runtime event** that disrupts normal flow. Exception handling keeps the program **robust**.

6.1 Exception Hierarchy

The root is `java.lang.Throwable`, which splits into `Error` and `Exception`. `Exception` further splits into `RuntimeException` (unchecked) and other **checked** exceptions.

```
                Throwable
               /         \
          Error          Exception
     (unchecked)        /         \
                RuntimeException  IOException, SQLException...
                (UNCHECKED)        (CHECKED)
```

- **Error:** serious problems the program **should not handle** (e.g. **OutOfMemoryError**, **StackOverflowError**).
- **Exception:** conditions a program **can and should** handle.

6.2 Checked vs Unchecked Exceptions

- **Checked exceptions:** verified at **compile time**; must be handled or declared with **throws** (e.g. **IOException, SQLException, ClassNotFoundException, FileNotFoundException, InterruptedException**).
- **Unchecked exceptions:** subclasses of **RuntimeException**, checked at **runtime** (e.g. **NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, ClassCastException, NumberFormatException, IllegalArgumentException**).

Basis	Checked	Unchecked
Checked at	**Compile time**	**Runtime**
Superclass	Exception (not Runtime)	**RuntimeException**
Handling	**Mandatory**	Optional
Examples	IOException, SQLException	NullPointerException, ArithmeticException

6.3 try, catch, finally

- `try` — encloses risky code.
- `catch` — handles a matching exception type.
- `finally` — **always executes** (cleanup), whether or not an exception occurs; the only escape is `System.exit()` or JVM crash.

```
try {
    int x = 10 / 0;            // throws ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("Always runs");
}
```

6.4 Multiple catch, Nested try, Multi-catch

- **Multiple catch:** several `catch` blocks; the **most specific** exception must come **before** the more general (**superclass last**), else a compile error.
- **Nested try:** a `try` inside another `try`.
- **Multi-catch (Java 7+):** one block handles several types: `catch (IOException | SQLException e)`.

- **try-with-resources (Java 7+):** auto-closes resources implementing **AutoCloseable**.

6.5 throw and throws

- `throw` — **explicitly throws** an exception **instance** (`throw new ...`); used **inside** a method.
- `throws` — **declares** that a method **may throw** checked exceptions; appears in the method **signature**.

MCQ Trap: `throw` throws **one** object; `throws` declares (possibly **many**, comma-separated) exception types. `finally` **always runs** even if `try`/`catch` has a `return`. A more-general catch placed **before** a specific one is a **compile-time error**.

6.6 User-Defined (Custom) Exceptions

Create a custom exception by **extending** `Exception` (checked) or `RuntimeException` (unchecked), then **throw** it.

```
class InsufficientFundsException extends Exception {
    InsufficientFundsException(String msg) { super(msg); }
}
```

7. Multithreading

A **thread** is the **smallest unit of execution**. **Multithreading** runs multiple threads **concurrently**, sharing the **same process memory**, for better CPU utilization and responsiveness.

7.1 Thread Life Cycle

The `Thread.State` enum defines **6 states**: **NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED**. Classic texts describe **5 states** (New, Ready/Runnable, Running, Blocked/Waiting, Dead).

```
NEW --start()--> RUNNABLE --scheduler--> RUNNING
      ^              |                      |
      |        (lock/wait/sleep)            | run() ends
      |              v                      v
      +-------- WAITING/BLOCKED        TERMINATED (DEAD)
```

7.2 Creating Threads

2 ways to create a thread:

- **Extend** `Thread` and override `run()`, then call `start()`.
- **Implement** `Runnable`, define `run()`, pass it to a `Thread` and call `start()`. Preferred because the class can still **extend** another class.

```
class T extends Thread { public void run(){ ... } }      // way 1
class R implements Runnable { public void run(){ ... } }  // way 2
```

MCQ Trap: Always call `start()` (not `run()`) to begin a new thread. Calling `run()` **directly** executes in the **current thread** — no new thread is created. Implementing **Runnable** is preferred over extending Thread (single inheritance limit).

7.3 Thread Methods

- `start()` — registers the thread and invokes `run()` on a **new** call stack.
- `run()` — contains the thread's task.
- `sleep(ms)` — **static**; pauses current thread; **does NOT release locks**; throws **InterruptedException**.
- `join()` — current thread **waits** for another to finish.
- `yield()` — **static**; hints the scheduler to pause the current thread for equal-priority threads.

- `interrupt()` — signals a thread to stop waiting/sleeping.

7.4 Thread Priorities

Priority ranges **1 to 10**: `MIN_PRIORITY = 1`, `NORM_PRIORITY = 5` **(default)**, `MAX_PRIORITY = 10`. Priority is only a **hint** to the scheduler — behaviour is **platform dependent**.

7.5 Synchronization

Synchronization prevents multiple threads from corrupting **shared data** (race conditions) by allowing only **one thread at a time** to hold an object's **monitor/lock**.

- **synchronized method:** locks the whole object (or class for static).
- **synchronized block:** locks a specified object for a code region (finer-grained).

7.6 Inter-Thread Communication

Methods `wait()`, `notify()`, `notifyAll()` are defined in the `Object` class (not Thread) and **must be called from a synchronized context**.

- `wait()` — releases the lock and waits until notified.
- `notify()` — wakes **one** waiting thread; `notifyAll()` wakes **all**.

> **MCQ Trap:** `wait()/notify()/notifyAll()` **belong to** `Object`, not `Thread`. `wait()` **releases the lock**, while `sleep()` **does NOT**. Both must run inside a **synchronized** block/method or throw **IllegalMonitorStateException**.

7.7 Deadlock

Deadlock occurs when two or more threads each **hold a lock** the other needs and **wait forever**. Prevention: acquire locks in a **consistent global order**, use **timeouts**, or avoid nested locks.

```
Thread-1 holds Lock-A, wants Lock-B
Thread-2 holds Lock-B, wants Lock-A     -> DEADLOCK (circular wait)
```

8. Event Handling & Applets

8.1 Event Handling

Event-driven programming: program flow is determined by **events** (clicks, key presses, mouse movement) rather than sequential code.

8.1.1 Delegation Event Model

The **Delegation Event Model** has **3 participants**: an **event Source** generates an event, an **event Object** describes it, and a **registered Listener** handles it. The source **delegates** handling to listeners registered via `addXxxListener()`.

```
[Source: Button] --fires--> [Event: ActionEvent] --notifies--> [Listener: ActionListener]
```

8.1.2 Event Classes & Listener Interfaces

Event classes live in `java.awt.event`; the root event class is `EventObject` (in `java.util`) -> `AWTEvent`.

Event Class	Listener Interface	Triggered by
ActionEvent	**ActionListener**	Button click, menu, Enter
MouseEvent	**MouseListener / MouseMotionListener**	Mouse click/move
KeyEvent	**KeyListener**	Keyboard key
WindowEvent	**WindowListener**	Window open/close
ItemEvent	**ItemListener**	Checkbox / list select

Event Class	Listener Interface	Triggered by
FocusEvent	**FocusListener**	Gain/lose focus

8.1.3 Adapter Classes

Adapter classes provide **empty (default) implementations** of listener interfaces with **many methods**, so you override **only the ones you need** (e.g. **MouseAdapter, KeyAdapter, WindowAdapter**). They reduce boilerplate code.

> **MCQ Trap:** Adapter classes exist for listeners with **multiple methods** (Mouse, Key, Window). `ActionListener` has **no adapter** because it has only **one** method. Listeners are registered with `addXxxListener()`.

8.2 Applets

An **applet** is a small Java program that runs **inside a browser or appletviewer**, not standalone, and has **no `main()` method**. Classic applets extend `java.applet.Applet` (AWT) or `javax.swing.JApplet` (Swing).

8.2.1 Applet Life Cycle

The applet life cycle has **5 methods**, called by the browser/viewer in order:

- `init()` — called **once**; initialization.
- `start()` — called after init and each time the applet becomes active.
- `paint(Graphics g)` — draws/redraws the applet's display.
- `stop()` — called when the applet is no longer visible.
- `destroy()` — called **once** before the applet is removed; cleanup.

```
LOADED:   init() -> start() -> paint()
INACTIVE: stop()                 (e.g. minimized)
ACTIVE:   start() -> paint()     (restored)
UNLOAD:   destroy()
```

> **MCQ Trap:** Order is **init -> start -> paint** (running) and **stop -> destroy** (closing). `init()` and `destroy()` run **once**; `start()`/`stop()` can run **many times**. Applets have **no `main()`**. (Applets were **deprecated in Java 9** and **removed in Java 17**, but remain in the IBPS syllabus.)

8.2.2 Applet vs Application

Basis	Applet	Application
Entry point	**No main()**; uses `init()`	`main()`
Execution	Runs in **browser/appletviewer**	Runs standalone via **JVM**
Base class	**Applet / JApplet**	Any class
Security	**Restricted (sandbox)**	Full access

8.2.3 AWT & Swing Basics

- **AWT (Abstract Window Toolkit, `java.awt`): heavyweight** components using **native OS peers**.
- **Swing (`javax.swing`): lightweight**, **pure-Java** components prefixed with **"J"** (JButton, JLabel, JFrame); part of **JFC**; supports **pluggable look-and-feel**.
- **Layout managers** arrange components: **FlowLayout** (left-to-right; default for **Panel/Applet**), **BorderLayout** (5 regions **North, South, East, West, Center**; default for **Frame/Window**), **GridLayout** (equal grid), **GridBagLayout** (flexible), **CardLayout**, **BoxLayout**.

Basis	AWT	Swing
Weight	**Heavyweight**	**Lightweight**

Basis	AWT	Swing
Peers	**OS native peers**	**Pure Java**
Package	**java.awt**	**javax.swing**
Components	Fewer	**Rich (J-components)**
Look & feel	OS-fixed	**Pluggable**

MCQ Trap: Default layout: **Frame -> BorderLayout**, **Panel/Applet -> FlowLayout**. Swing components start with **"J"** and are **lightweight**; AWT components are **heavyweight**. BorderLayout has **5** regions.

8.3 I/O Streams (Overview)

Java I/O is **stream-based**. A **stream** is a sequence of data. Two categories:

- **Byte streams:** read/write **8-bit bytes**; root classes `InputStream` and `OutputStream` (e.g. **FileInputStream, FileOutputStream**). Use for binary data.
- **Character streams:** read/write **16-bit Unicode characters**; root classes `Reader` and `Writer` (e.g. **FileReader, FileWriter, BufferedReader**). Use for text.
- **File handling:** the `File` class represents a path; combine with **FileReader/FileWriter** or **FileInputStream/FileOutputStream**. `System.in` (InputStream), `System.out`/`System.err` (PrintStream) are standard streams.

MCQ Trap: Byte streams = InputStream/OutputStream (binary); **Character streams = Reader/Writer** (text, Unicode). `Scanner` and `BufferedReader` are common for console input.

9. Quick Revision / Exam Pointers

- **4 pillars of OOP: Abstraction, Encapsulation, Inheritance, Polymorphism** (A-P-I-E).
- **Overloading vs Overriding:** compile-time/same-class/different-params **vs** runtime/two-classes/same-signature.
- **Abstract class vs Interface:** partial vs full abstraction; **extends one** vs **implements many**; interface variables are **public static final**.
- **Access modifiers (widening): private -> default -> protected -> public**.
- **Exception hierarchy: Throwable -> {Error, Exception -> RuntimeException}**; checked = compile-time, unchecked = runtime.
- **Thread states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED** (6).
- **Thread priorities: MIN 1, NORM 5, MAX 10**.
- **Applet life cycle: init -> start -> paint -> stop -> destroy**.
- `final` / `finally` / `finalize()` are distinct concepts.
- **8 primitive types**; **char = 2 bytes**; Java is **pass-by-value** only.
- **Java does not support:** multiple inheritance of classes, operator overloading, pointers, global variables, `goto` (reserved but unused), destructors, built-in copy constructor.

Memory Trick — Applet life cycle: "**I S P S D**" = **I**nit, **S**tart, **P**aint, **S**top, **D**estroy. The bookends **init** and **destroy** run **once**.

Shortcut: When a question says "**which runs first / how many times**", recall: **static block** (class load) -> **constructor** (object) per object; **init/destroy once**, **start/stop many**; **finally always**.